CampusLaundry Management System

Stored Procedures Specification & Server-Side Business Logic

Project: CampusLaundry Management System	**Category:** Database Stored Procedures	**DBMS Engine:** MySQL 8.0+ / InnoDB
Schema Normalization: 3NF (Strict Third Normal Form)	**Academic Scope:** Database Management Systems Laboratory	**Deployment:** Cloud Production Verified (Aiven)

1. Stored Procedures Architecture & Design Philosophy

Stored procedures in CampusLaundry encapsulate complex query plans and business logic directly inside the MySQL database engine. This architectural choice delivers three decisive advantages:

1. **Zero Network Latency Chatter:** Consolidates multiple queries into a single client-server round trip.
2. **Security & Parameter Sanitization:** Strong typing protects against SQL injection vulnerabilities.
3. **Optimized Execution Plans:** MySQL compiles and caches the procedural execution plan in memory for sub-second responses.

2. Procedure 1: sp_GetStudentPortalData

Signature: sp_GetStudentPortalData(IN p_std_id VARCHAR(20))
Purpose: Single-trip API backend driver for the Student Portal. Simultaneously dispatches two distinct result sets: (1) Card analytics joined from modular views, and (2) The 10 most recent laundry cycles with resolved kiosk and contractor data.

```
DELIMITER //

DROP PROCEDURE IF EXISTS sp_GetStudentPortalData //

CREATE PROCEDURE sp_GetStudentPortalData(
    IN p_std_id VARCHAR(20)
)
BEGIN
    -- Result Set 1: Student Overview & Analytics Cards (from vw_student_portal_overview)
    SELECT
        std_id,
        std_name,
        contact,
        hostel,
        room,
        academic_year,
        quota,
        cycle_used,
        cycle_remaining,
        burn_percentage,
        pace_status,
        active_transaction_id,
        active_status,
        active_total_weight,
        active_drop_point_id,
        active_drop_point_location,
        active_drop_off_date,
        active_expected_completion_date
    FROM vw_student_portal_overview
    WHERE std_id = p_std_id;

    -- Result Set 2: Recent 10 Laundry Orders with Drop Point & Vendor Resolution
    SELECT
        t.Transaction_ID,
        t.std_id,
        t.Drop_Point_ID,
        dp.Location AS drop_point_location,
        dp.Vendor_ID,
        v.Vendor_Name AS vendor_name,
        t.Drop_Off_Date,
        t.Expected_Completion_Date,
        t.Total_Weight,
        t.Cycles_Deducted,
        t.Status
    FROM laundry_transaction t
    JOIN drop_point dp ON t.Drop_Point_ID = dp.Drop_Point_ID
    JOIN vendor v ON dp.Vendor_ID = v.Vendor_ID
    WHERE t.std_id = p_std_id
    ORDER BY t.Drop_Off_Date DESC, t.Transaction_ID DESC
    LIMIT 10;
END //

DELIMITER ;
```

Client Execution Syntax:

```
CALL sp_GetStudentPortalData('2502050680');
```

3. Procedure 2: sp_GetHostelDashboardMetrics

Signature: sp_GetHostelDashboardMetrics()
Purpose: Computes campus-wide executive Key Performance Indicators (KPIs) in a single ultra-fast pass using **conditional aggregation** (COUNT(CASE WHEN ... THEN 1 END)) combined with an isolated CROSS JOIN. This avoids cartesian product explosions and table full-scans.

```
DELIMITER //

DROP PROCEDURE IF EXISTS sp_GetHostelDashboardMetrics //

CREATE PROCEDURE sp_GetHostelDashboardMetrics()
BEGIN
    SELECT
        t.total_transactions,
        t.active_loads,
        t.completed_loads,
        t.avg_load_kg,
        s.total_students,
        s.total_cycles_used,
        s.total_campus_quota,
        dp.active_kiosks
    FROM (
        -- Subquery 1: Transaction Volume & Throughput Aggregates
        SELECT
            COUNT(*) AS total_transactions,
            COUNT(CASE WHEN Status NOT IN ('Completed', 'Cancelled') THEN 1 END) AS active_loads,
            COUNT(CASE WHEN Status = 'Completed' THEN 1 END) AS completed_loads,
            ROUND(AVG(Total_Weight), 2) AS avg_load_kg
        FROM laundry_transaction
    ) t
    CROSS JOIN (
        -- Subquery 2: Student Population & Campus Quota Balance
        SELECT
            COUNT(*) AS total_students,
            COALESCE(SUM(cycle_used), 0) AS total_cycles_used,
            COALESCE(SUM(quota), 0) AS total_campus_quota
        FROM students
    ) s
    CROSS JOIN (
        -- Subquery 3: Active Drop Point Infrastructure Count
        SELECT
            COUNT(*) AS active_kiosks
        FROM drop_point
        WHERE Operational_Status = 'Active'
    ) dp;
END //

DELIMITER ;
```

Client Execution Syntax:

```
CALL sp_GetHostelDashboardMetrics();
```

4. Procedure 3: sp_AuthenticateAdmin

Signature: sp_AuthenticateAdmin(IN p_username VARCHAR(30))
Purpose: Secure credential resolution for administrative authentication. Safely pulls salted bcrypt hashes and role metadata for active users, shielding administrative user records from blind query injection.

```
DELIMITER //

DROP PROCEDURE IF EXISTS sp_AuthenticateAdmin //

CREATE PROCEDURE sp_AuthenticateAdmin(
    IN p_username VARCHAR(30)
)
BEGIN
    SELECT
        admin_id,
        username,
        password_hash,
        full_name,
        role,
        is_active,
        last_login
    FROM admin_users
    WHERE username = p_username
      AND is_active = TRUE;
END //

DELIMITER ;
```

Client Execution Syntax:

```
CALL sp_AuthenticateAdmin('warden_b1');
```

5. Stored Procedure Metadata & Administrative Commands

```
-- 1. Inspect Procedure Metadata in Catalog
SHOW PROCEDURE STATUS WHERE Db = 'laundry';

-- 2. Inspect Exact Stored Procedure Definition
SHOW CREATE PROCEDURE sp_GetStudentPortalData;

-- 3. Query Routine Parameters from Information Schema
SELECT
    SPECIFIC_NAME,
    ROUTINE_TYPE,
    DTD_IDENTIFIER,
    CREATED,
    LAST_ALTERED
FROM information_schema.ROUTINES
WHERE ROUTINE_SCHEMA = 'laundry';
```